

Table of Contents

- [1. System Overview](#)
- [2. Technology Stack](#)
- [3. Architecture Design](#)
- [4. System Flow](#)
- [5. Database Design](#)
- [6. Embedding Pipeline](#)
- [7. File Structure Reference](#)
- [8. API Endpoints](#)
- [9. Error Handling Strategy](#)

Introduction

I encountered a permission error while trying to automatically insert this into your Google Doc, but I have drafted the introduction for you based on your thought process and the provided documentation. You can copy and paste this directly into the `## Introduction` section of your report:

Introduction

The Problem The core problem addressed by this system is the need for a fully local, self-contained service capable of ingesting PDF documents and performing highly accurate searches across their contents without relying on external cloud APIs or GPU hardware.

My Approach and Thought Process

To solve this problem, I approached the system design in four logical phases:

1. **Data Flow & Core Logic:** First, I conceptualized the data lifecycle. During the ingest phase, a user uploads a PDF, which is extracted and goes to a chunker to be split into manageable overlapping pieces. Each chunk is saved in the database while simultaneously being transformed into a vector embedding, which is also stored. For the search phase, the flow is similar but involves transforming

the user's query into an embedding rather than storing new data. To retrieve the most relevant data, I researched retrieval techniques and decided to implement a Hybrid Search approach. This combines semantic vector similarity search with PostgreSQL full-text keyword search, merging the results using Reciprocal Rank Fusion (RRF) to return the most accurate top results.

2. **Data Modeling & Software Architecture:** Second, I designed the database schema, carefully defining the attributes and relationships for the `documents` and `chunks` tables. After establishing the data model, I considered the software design. While I initially thought about a standard MVC pattern, I structured the application using a strict 4-layer architecture consisting of Routes, Controllers, Services, and Repositories. This separation of concerns ensures that HTTP routing, business logic orchestration, domain operations (like chunking and embedding), and database CRUD operations are cleanly isolated.
3. **Infrastructure & Deployment:** Third, I focused on the infrastructure. I selected a robust technology stack including FastAPI, PostgreSQL with the pgvector extension, and FastEmbed for local ONNX inference. I then containerized the entire application using Docker and created a Docker Compose orchestration file to seamlessly coordinate the application container and the database container.
4. **Robustness, Patterns & Optimization:** Finally, after writing the core code, I focused on edge cases and system robustness. I implemented strategies to bypass issues, such as using SHA-256 hashing on the raw PDF bytes to handle duplicate document uploads. I applied object-oriented programming and design patterns, specifically using a Factory and Interface pattern to manage the embedding provider for future extensibility. I also optimized the system by implementing an in-memory background worker queue (`asyncio.Queue`) to decouple HTTP requests from heavy processing tasks. I retained `asyncio.to_thread` to ensure the synchronous embedding model does not block the async event loop, and I ensured observability through centralized error handling and request metrics middleware.

1. System Overview

Purpose

This system provides a self-contained, containerized service for:

- 1. **Ingesting** PDF documents — extracting text, splitting into chunks, and generating vector embeddings.
- 2. **Searching** across ingested documents — using hybrid semantic and keyword search to find the most relevant text passages.

Core Workflow

PDF Upload → Text Extraction → Chunking → Embedding → Storage → Searchable

The system is fully local — it does not depend on any external APIs, cloud services, or GPU hardware. Everything runs inside two Docker containers: the FastAPI application and PostgreSQL with pgvector.

2. Technology Stack

Core Framework

Technology	Role	Why This Choice
FastAPI	Web framework	Native async, auto-generated docs, Pydantic validation

Database

Technology	Role	Why This Choice
PostgreSQL	Primary database	Battle-tested relational database
pgvector	Vector extension	Vector similarity search without a separate vector DB

Technology	Role	Why This Choice
SQLAlchemy	ORM	Async support, mature migration tooling
asyncpg	Driver	High-performance async PostgreSQL driver
Alembic	Migrations	Version-controlled, reproducible schema changes

AI / NLP

Technology	Role	Why This Choice
FastEmbed	Vector embeddings	Local ONNX inference, no API keys required
BAAI/bge-small-en-v1.5	Embedding model	Lightweight (384-dim), good quality, runs on CPU
LangChain Text Splitters	Text chunking	Recursive, semantically-aware splitting
PyMuPDF (fitz)	PDF parsing	Fast, reliable, in-memory parsing

Infrastructure

Technology	Role	Why This Choice
Docker,Docker Compose	Containerization	Consistent, reproducible deployments,Multi-contain er coordination
Pydantic Settings	Configuration	Type-safe environment variable loading

3. Architecture Design

Layered Architecture



The application follows a strict **4-layer architecture** where each layer has a single responsibility and only communicates with its adjacent layer:

Layer	Description	Components / Services
Routes Layer	HTTP endpoints, request validation, response schemas	FastAPI Web Framework
Controllers Layer	Business logic orchestration, error handling	Ingest & Search Controllers
Services Layer	Domain operations: PDF parsing, chunking, embedding, hashing, vector search	FastEmbed (ONNX), PyMuPDF
Repositories Layer	Database CRUD, SQL queries, ORM mapping	PostgreSQL + pgvector

Layer Responsibilities

Layer	Files	Responsibility
Routes	<code>routes/base.py</code> , <code>routes/ingest.py</code> , <code>routes/search.py</code>	HTTP endpoint definitions, request/response validation, dependency injection
Controllers	<code>controllers/ingest_controller.py</code> , <code>controllers/search_controller.py</code>	Business logic orchestration, multi-step workflows, error aggregation

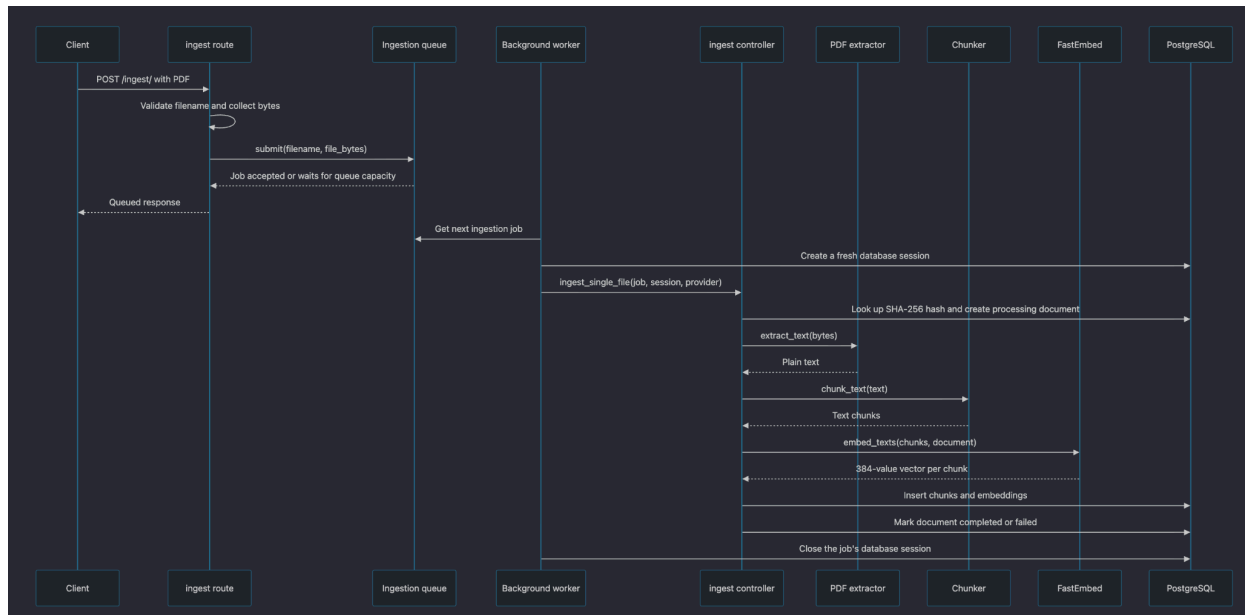
Layer	Files	Responsibility
Services	<code>services/pdf_extractor.py,</code> <code>services/chunker.py,</code> <code>services/hashing.py,</code> <code>services/vector_search.py</code>	Single-purpose domain operations
Repositories	<code>repositories/document_repository.py,</code> <code>repositories/chunk_repository.py</code>	Database CRUD operations, SQL queries

Startup Sequence

1. **orchestrate.sh --action start** triggers Docker Compose.
2. **Start PostgreSQL container:** Runs `init-db.sh` to enable extensions and passes health checks.
3. **Start App container:** Runs Alembic migrations and initializes the FastAPI lifespan (loading models).
4. **Ready State:** The system polls the health endpoint until port 8000 is responsive.

4. System Flow

4.1 Ingestion Flow



The ingestion process handles the transformation of raw PDF bytes into searchable vector data.

Phase 1: HTTP Request (Queueing)

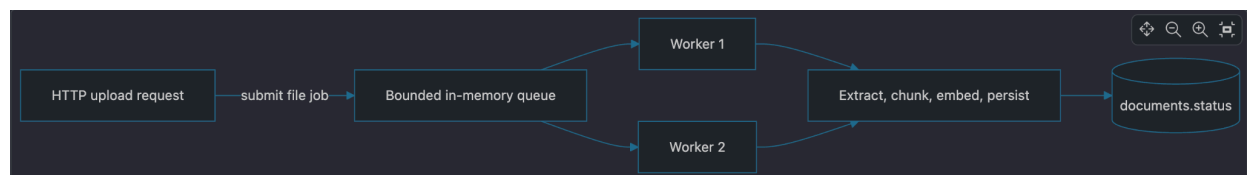
1. **POST Request & Validation:** The client issues a `POST /ingest/` call payload containing PDF data to the ingestion endpoint, which verifies document parameters and extracts raw content bytes.
2. **Queue Submission:** Next, the route pushes task details alongside payload data into the internal task queue, backpressuring if system capacity limits are met.
3. **Queued Response:** Upon successful enqueueing, the system promptly responds with an asynchronous job acknowledgement back to the caller.

Phase 2: Background Worker Processing

1. **Worker Activation & DB Session:** An asynchronous background processor polls pending items from the queue and instantiates a dedicated database connection context.
2. **Controller Delegation:** The worker delegates execution logic to the controller module by invoking `ingest_single_file(job, session, provider)`.

3. **Hashing & Document Record:** Deduplication is handled via cryptographic SHA-256 checks against existing records before persisting an initial document tracking status.
4. **Text Extraction:** PDF extraction services analyze binary streams to retrieve unformatted body text.
5. **Chunking:** The application breaks extracted text down into uniform, overlapping segments using text splitting utilities.
6. **Embedding:** Generated chunks pass to FastEmbed engine instances to produce corresponding 384-dimensional dense vectors.
7. **Chunk Storage:** Text segments alongside calculated embeddings are written to persistent PostgreSQL tables.
8. **Completion:** Document entity statuses update within the database to reflect terminal state outcomes.

Session Cleanup: Finally, worker routines release active relational database connections back to available connection pools.



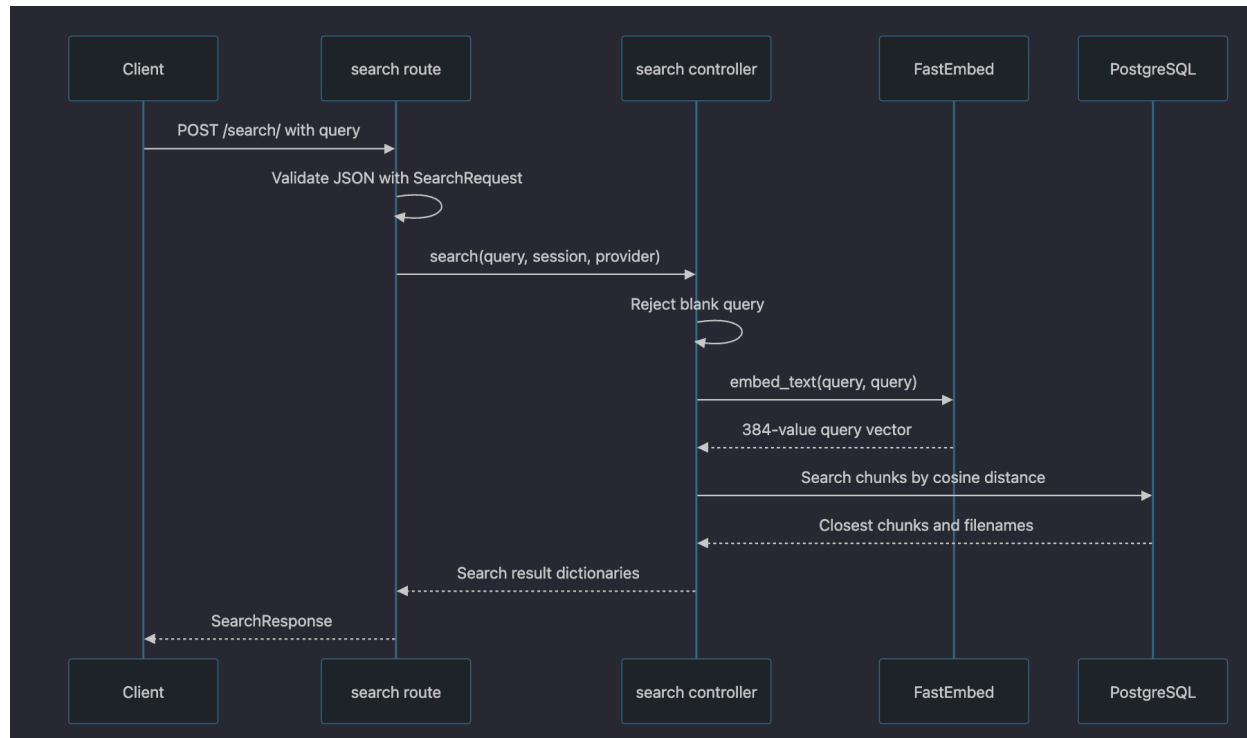
When a user uploads a PDF, the system simply checks the file and adds it to a queue instead of making the user wait for the heavy processing (like text extraction and AI embedding) to finish. The user immediately gets a "Queued" response, while background workers pull jobs from the queue to process them behind the scenes.

You can control how many files process at the exact same time using the `INGESTION_WORKERS` setting. The `INGESTION_QUEUE_SIZE` dictates how many files can wait in line. If the line fills up completely, new uploads will have to wait for space to free up before they can join the queue.

It's important to note that this queue lives entirely in the specific server's memory. If you have multiple server replicas running, each one manages its own separate queue, workers, and AI models. This means the total maximum concurrency across your whole setup is simply the number of server replicas multiplied by the number of workers per server.

Because the queue is just held in short-term memory, it isn't durable—if the server crashes or restarts, any pending jobs waiting in line will be lost. For a heavy-duty production environment, you would want to replace this internal memory queue with a dedicated external message broker, like Redis, RabbitMQ, or Azure Service Bus.

4.2 Search Flow



Search leverages hybrid retrieval to ensure both keyword accuracy and semantic relevance.

1. **Validation** — Reject empty or whitespace-only queries.
2. **Query embedding** — Convert the query string into a 384-dim vector using the same model.
3. **Hybrid search** — Execute parallel retrieval strategies:
 - **Semantic** — pgvector cosine distance against stored chunk embeddings.
 - **Lexical** — PostgreSQL full-text search (`ts_rank_cd` with `websearch_to_tsquery`).
 - **Fusion** — Reciprocal Rank Fusion (RRF) merges both rankings.

4.3 Hybrid Search Algorithm

The search uses **Reciprocal Rank Fusion (RRF)** to combine signals:

```
hybrid_score = 1/(rrf_k + semantic_rank) + 1/(rrf_k + lexical_rank)
```

- `rrf_k` is a constant (default 60) that controls rank weighting.
- Final results are filtered by a minimum relevance score and limited to `TOP_K`.

5. Database Design

Entity Relationship

The schema focuses on a 1:N relationship between Documents and their constituent Chunks.

- **documents**: Stores metadata including `file_hash` (unique), status, and processing timestamps.
- **chunks**: Stores the actual `chunk_text`, the `VECTOR(384)` embedding, and a foreign key to the parent document.

Key Design Points

- **UUID primary keys** — Generated by the application for consistency.
- **SHA-256 deduplication** — Indexed for fast lookup to prevent redundant processing.
- **JSONB metadata** — Flexible storage for future extensibility (e.g., page numbers).

6. Embedding Pipeline

Provider Architecture

The embedding system uses a **factory + interface** pattern:

- `EmbeddingInterface` defines the contract.
- `LocalProvider` uses FastEmbed (ONNX) for local inference.
- `EmbeddingProviderFactory` handles provider instantiation based on environment variables.

Current Implementation: LocalProvider

- **Model:** `BAAI/bge-small-en-v1.5` (384-dimensional).
- **Runtime:** ONNX on CPU.
- **Concurrency:** Managed via a background worker queue. The `INGESTION_WORKERS` setting controls how many files process simultaneously, and `INGESTION_QUEUE_SIZE` limits the waiting backlog to protect system memory. `asyncio.to_thread` is still used to prevent the synchronous embedding model from blocking the main application.

7. File Structure Reference

- **Root:** Orchestration scripts, API specifications (`swagger.yaml`), and main documentation.
- **docker/:** Deployment configurations, Dockerfile, and environment templates.
- **src/:** Core logic including FastAPI routes, business controllers, and data repositories.
- **evaluation/:** Benchmarking scripts for RAG quality.
- **test/:** Integration test suite.

8. API Endpoints

GET /api/v1/

Welcome / Health Check

Returns application name and version.

POST /api/v1/ingest/

Ingest PDF Documents

Accepts PDF files via multipart upload or a directory path string.

- **Response:** Returns success count and list of filenames processed.

POST /api/v1/search/

Semantic Search

Searches ingested documents using hybrid retrieval.

- **Request:** JSON containing the search query string.
- **Response:** A ranked list of matching text passages with relevance scores.

9. Error Handling Strategy

The system uses a centralized error handling strategy to map custom exceptions (like `PDFExtractionError`) to standard HTTP status codes.

- **Safe Public Messages:** Internal stack traces are logged but not exposed to users.
- **Partial Failure:** During batch ingestion, one failing file does not halt the entire process.
- **Transaction Safety:** Database operations use rollback logic to prevent orphaned records.